

COMPSYS302

Design: Software Practice

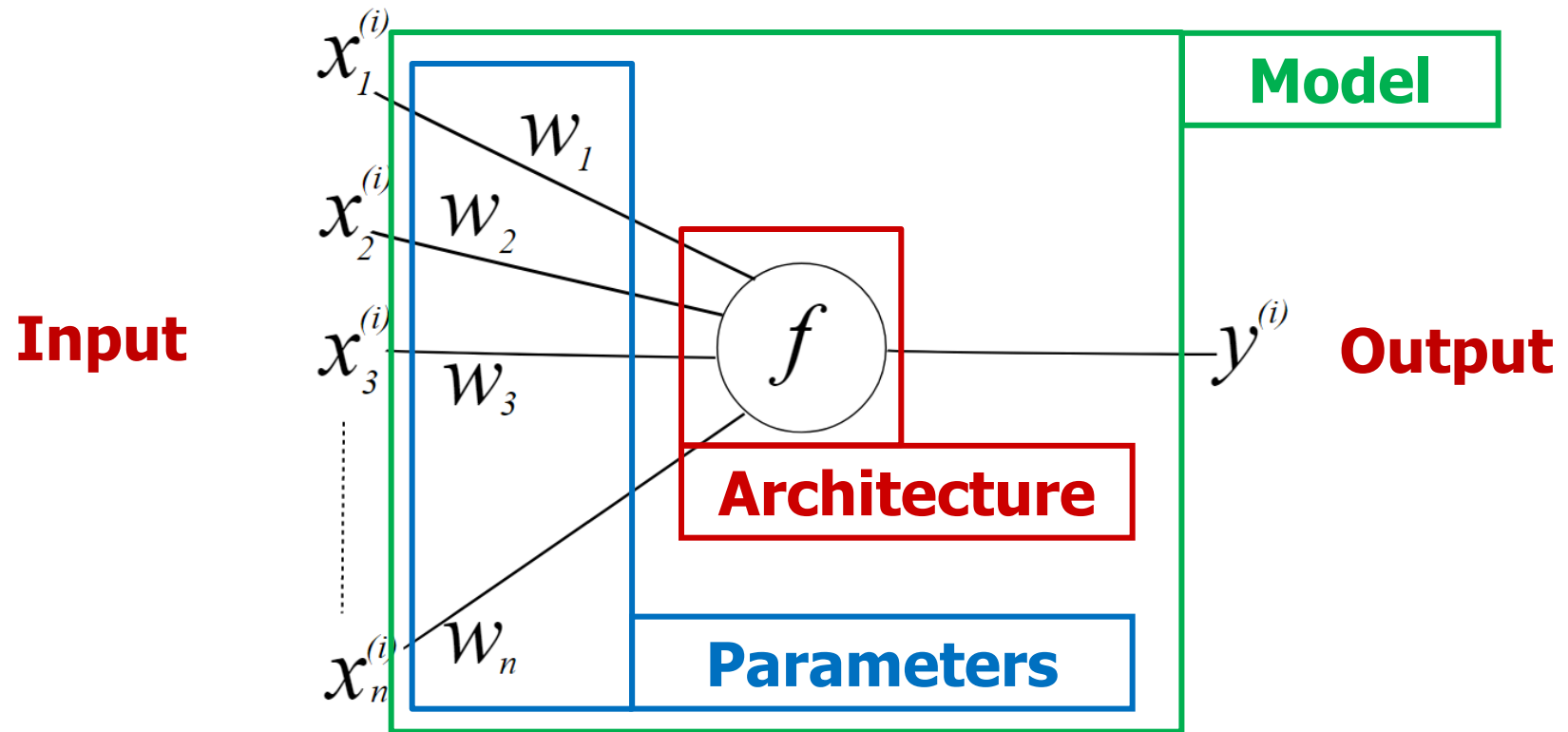
Introduction to AI #5 – Dataset

Ho Seok AHN

hs.ahn@auckland.ac.nz

1. Neural Network

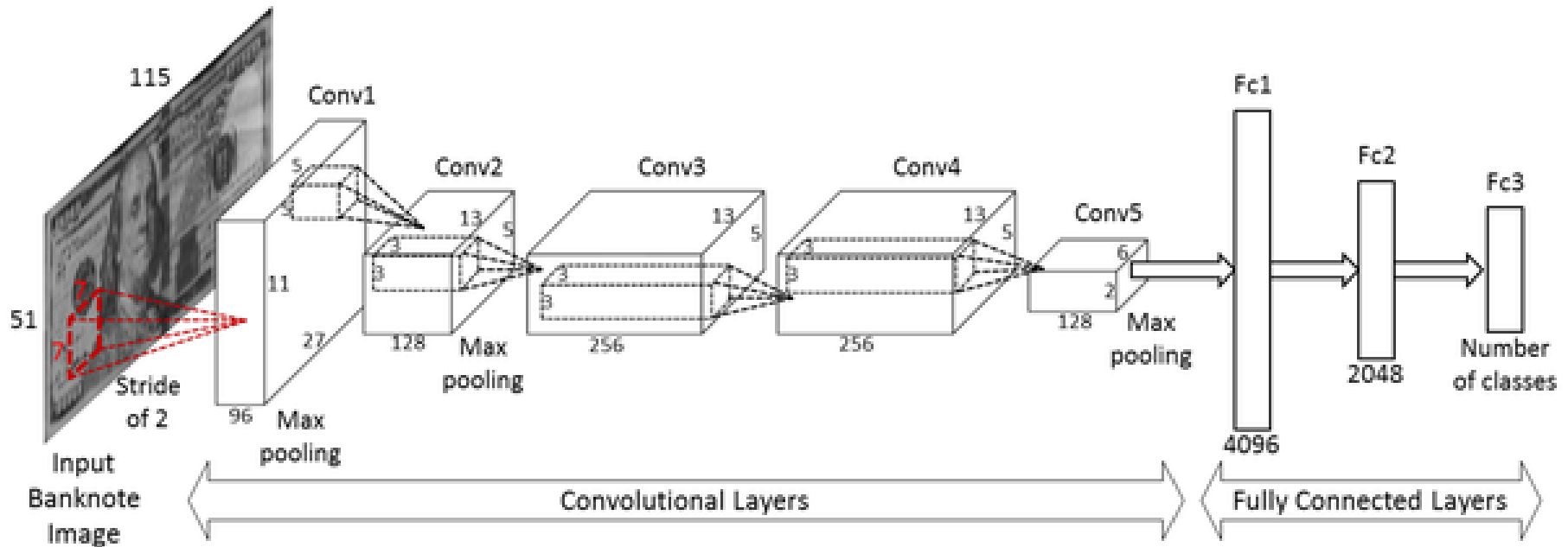
□ What is Model?



Model = Architecture + Parameters

2. Concept of CNN

□ CNN



- How deep is **convolutional layers**?
- **Size of kernel matrix** for convolutions?
- Which **polling** methods?
- Which **activation** functions?

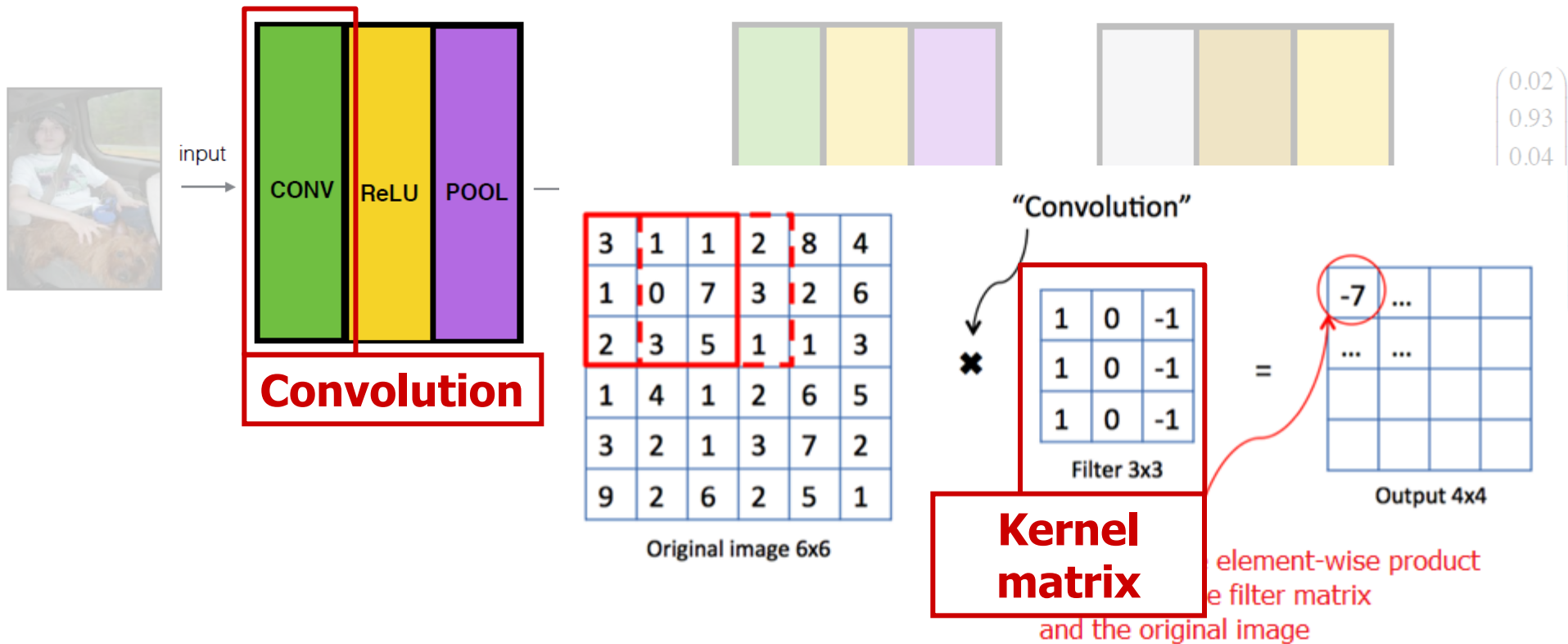
...



**Makes
different
models**

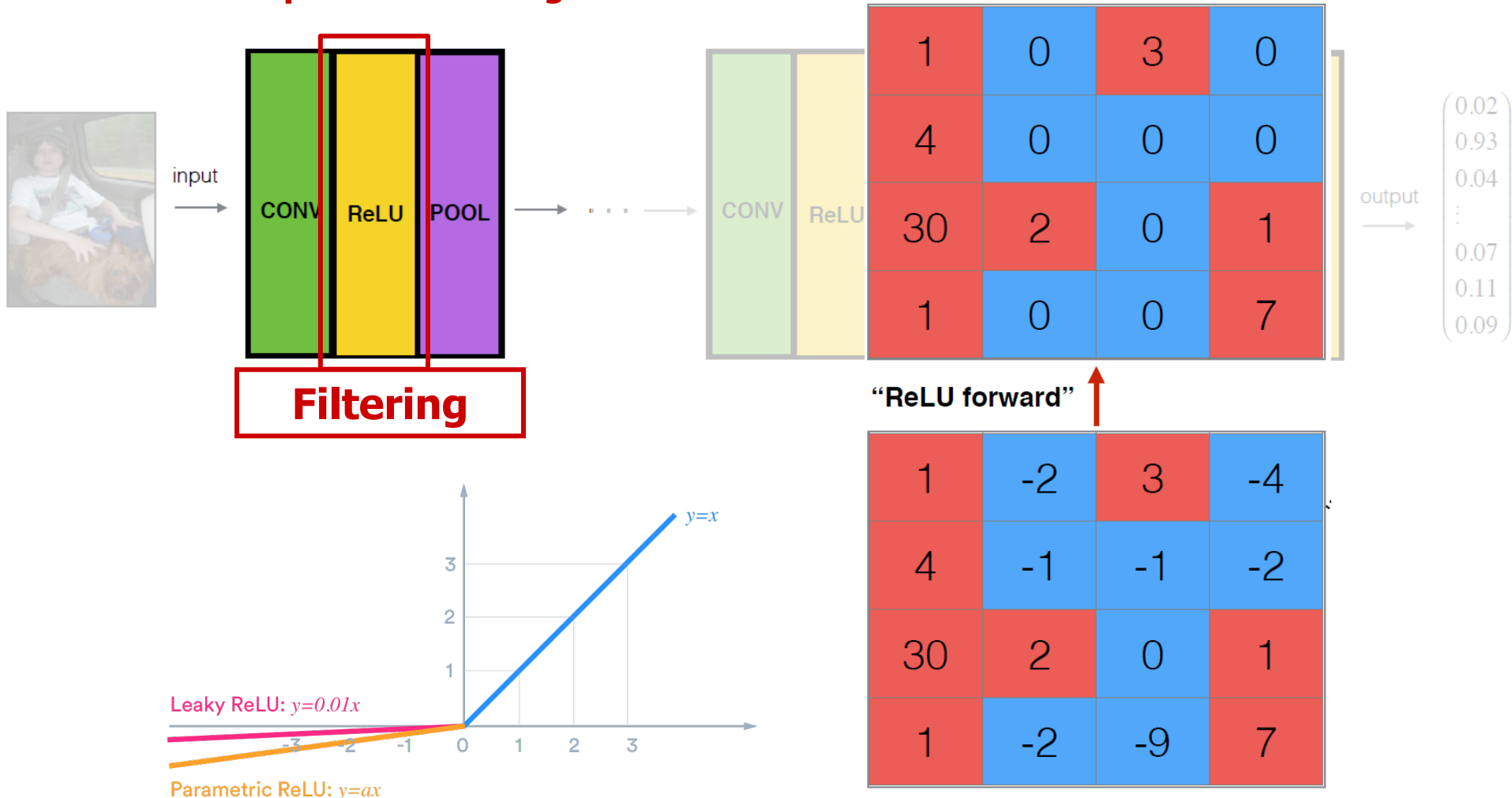
2. Concept of CNN

□ Recap - CNN Convolution



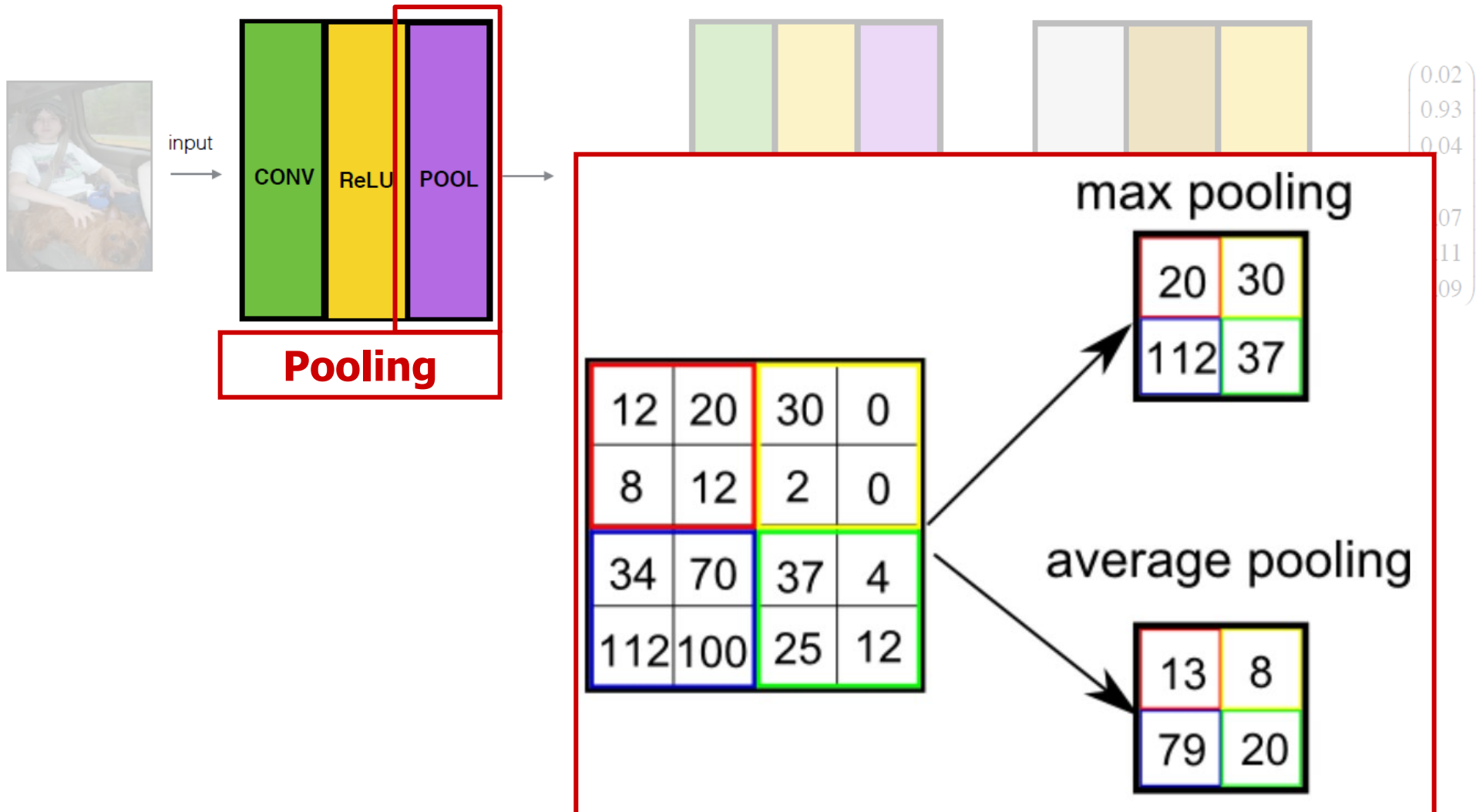
2. Concept of CNN

□ Recap - CNN Filtering



2. Concept of CNN

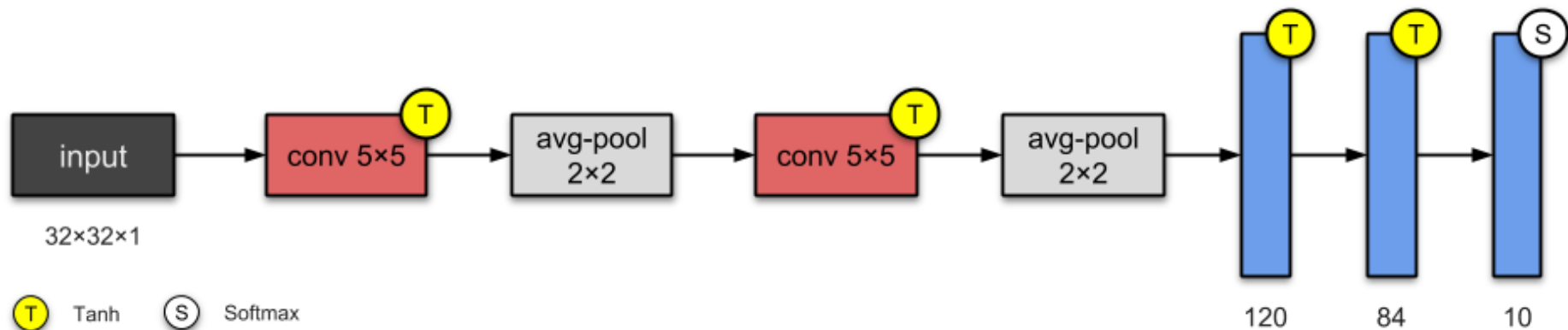
□ Recap - CNN Polling



3. Different models for CNN

□ LeNet-5 (1998)

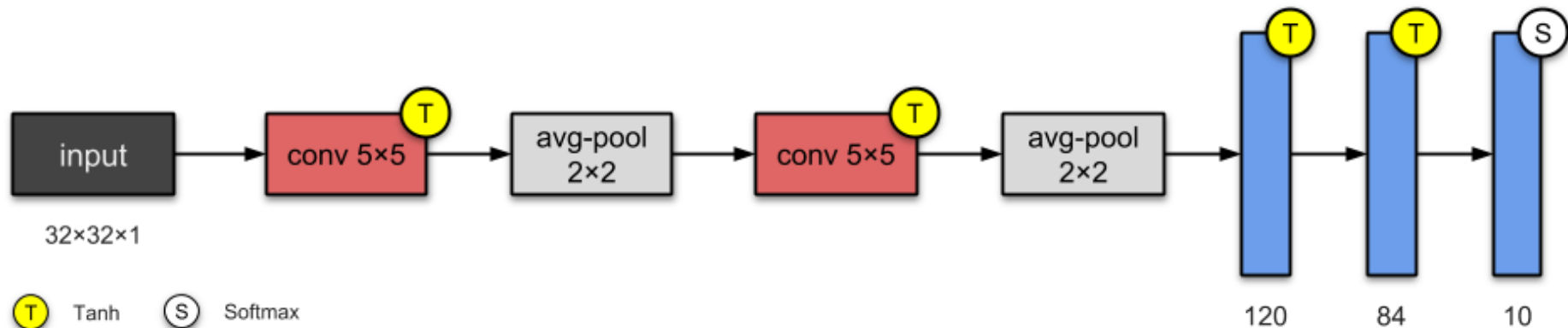
- one of the **simplest** architectures.
 - has 3 convolutional and 2 sub-sampling layers. → named as “5”
 - has about 60K parameters.
 - The average-pooling layer as we know it now was called a “sub-sampling layer”.
- This architecture has become the **standard ‘template’**: **stacking convolutions and pooling layers, and ending the network with one or more fully-connected layers.**



3. Different models for CNN

□ LeNet-5 (1998)

Layer		Feature Map	Size	Kernel Size	Stride	Activation
Input	Image	1	32x32	-	-	-
1	Convolution	6	28x28	5x5	1	tanh
2	Average Pooling	6	14x14	2x2	2	tanh
3	Convolution	16	10x10	5x5	1	tanh
4	Average Pooling	16	5x5	2x2	2	tanh
5	Convolution	120	1x1	5x5	1	tanh
6	FC	-	84	-	-	tanh
Output	FC	-	10	-	-	softmax



3. Different models for CNN

□ LeNet-5 (1998)

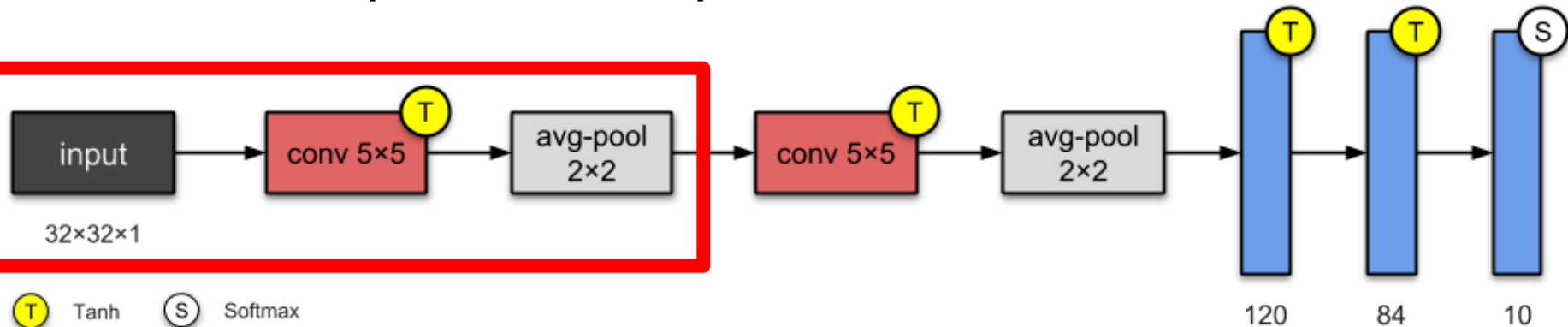
Layer		Feature Map	Size	Kernel Size	Stride	Activation
Input	Image	1	32x32	-	-	-
1	Convolution	6	28x28	5x5	1	tanh
2	Average Pooling	6	14x14	2x2	2	tanh

□ **Conv1**: 6 feature maps (i.e. Edge detection (horizontal, vertical, diagonal), Blob detection),
5×5 kernels, stride 1

□ **Input**: 32×32 grayscale images → **Output**: 28×28×6

□ **Avg Pooling1**: 2×2 kernels, stride 2

□ **Input**: 28×28×6 → **Output**: 14×14×6



3. Different models for CNN

□ LeNet-5 (1998)

	Layer	Feature Map	Size	Kernel Size	Stride	Activation
3	Convolution	16	10x10	5x5	1	tanh
4	Average Pooling	16	5x5	2x2	2	tanh

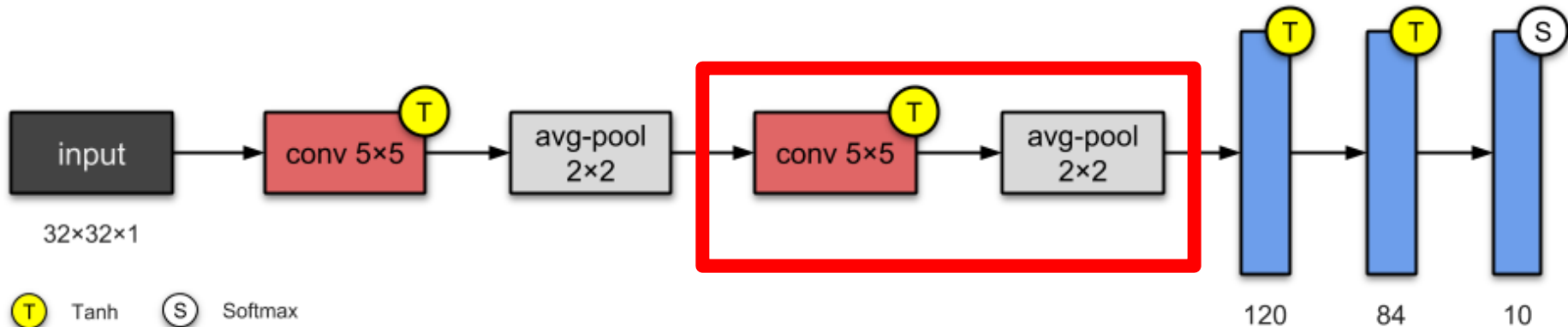
□ **Conv2:** 16 feature maps, 5×5 kernels, stride 1

□ **Input:** $14 \times 14 \times 6 \rightarrow$ **Output:** $10 \times 10 \times 16$

□ **Avg Pooling2:** 2×2 kernels, stride 2

□ **Input:** $10 \times 10 \times 16 \rightarrow$ **Output:** $5 \times 5 \times 16$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	X				X	X	X			X	X	X	X		X	X
1	X	X				X	X	X			X	X	X	X		X
2	X	X	X				X	X	X			X		X	X	X
3		X	X	X			X	X	X	X			X		X	X
4			X	X	X			X	X	X	X		X	X		X
5				X	X	X			X	X	X	X		X	X	X



3. Different models for CNN

□ LeNet-5 (1998)

	Layer	Feature Map	Size	Kernel Size	Stride	Activation
5	Convolution	120	1x1	5x5	1	tanh
6	FC	-	84	-	-	tanh
Output	FC	-	10	-	-	softmax

□ **Conv3**: 120 feature maps (i.e. Intersection patterns), $5 \times 5 \times 16$ kernels, stride 1

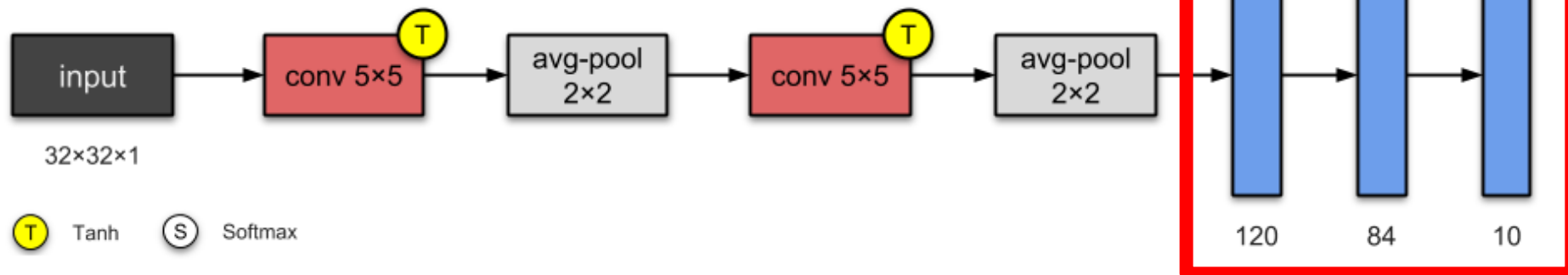
□ **Input**: $5 \times 5 \times 16 \rightarrow$ **Output**: $1 \times 1 \times 120$

□ **FC1**: Feedforward NN with 84 units

□ **Input**: $1 \times 1 \times 120 \rightarrow$ **Output**: 84

□ **FC2**: Feedforward NN with 10 units

□ **Input**: 84 $\rightarrow$ **Output**: 10



4. Dataset for CNN

☐ What are datasets for image classification?

- ☐ Image classification datasets consist of images and labels.
- ☐ Labels define which class(es) each image belongs to.
- ☐ Each image should have a label since this is a supervised learning task.
- ☐ The labels must be accurate, otherwise the model will pick up the errors in the dataset.

4. Dataset for CNN

☐ **Dataset types**

☐ **Binary Classification**

- ☐ Two possible classes
- ☐ Model output: Single probability (sigmoid activation).
- ☐ Example: cat vs. dog

☐ **Multiclass Classification**

- ☐ More than two mutually exclusive classes
- ☐ Model output: One probability per class (softmax activation).
- ☐ Example: Handwritten digits (1,2,3,4,5,6,7,8,9,0)

☐ **Multilabel Classification**

- ☐ Multiple independent labels per sample
- ☐ Model output: Multiple probabilities (sigmoid per class).
- ☐ Example: Image tagging (person, female, Kiwi)

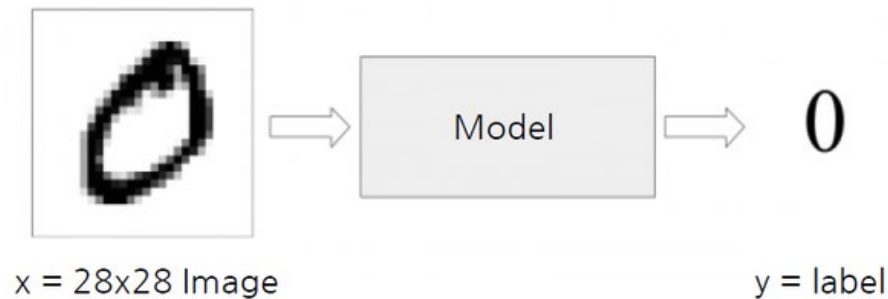
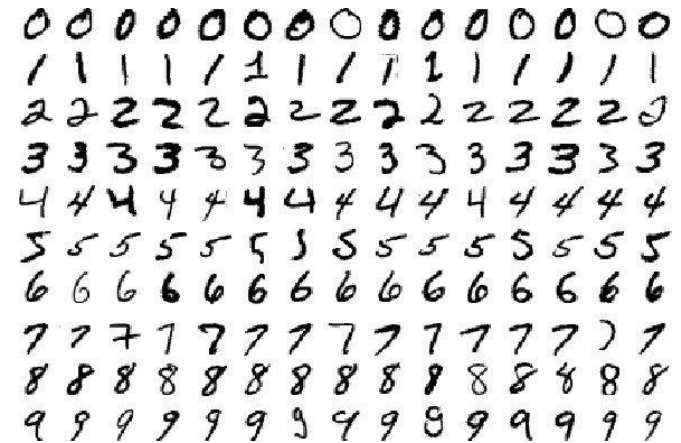
4. Dataset for CNN

□ Dataset Example

□ Number recognition: MNIST

□ Dataset have 0~9 number images

□ $28 \times 28 = 784$ pixels per image



4. Dataset for CNN

□ **ImageNet**

□ **Multi-class dataset**

□ **Large-Scale Dataset**

- Contains 14M+ labelled images across 20,000+ categories.

- Common benchmark for image classification tasks.

□ **Standard Subset**

for Training & Testing

- ImageNet-1K: 1.28M training images, 50K validation images, 1,000 classes.
- ImageNet-21K: A larger version with 21,841 categories.



4. Dataset for CNN

☐ **ImageNet**

☐ **ImageNet Challenge (ILSVRC)**

- ☐ ImageNet Large Scale Visual Recognition Challenge (ILSVRC) held annually (2010–2017).
- ☐ Led to breakthroughs in deep learning, e.g., AlexNet (2012), ResNet (2015), ViT (2020).

☐ **Dataset Properties**

- ☐ Images are high-resolution and diverse (varied lighting, backgrounds).
- ☐ Classes include animals, objects, scenes, and tools.

☐ **Labelling Process**

- ☐ Images in ImageNet are labelled using WordNet synsets (groups of synonyms representing a concept).
- ☐ Each image belongs to a single synset (class).

4. Dataset for CNN

☐ **Dataset types**

☐ **Binary Classification**

- ☐ Two possible classes
- ☐ Model output: Single probability (sigmoid activation).
- ☐ Example: cat vs. dog

☐ **Multiclass Classification**

- ☐ More than two mutually exclusive classes
- ☐ Model output: One probability per class (softmax activation).
- ☐ Example: Handwritten digits (1,2,3,4,5,6,7,8,9,0)

☐ **Multilabel Classification**

- ☐ Multiple independent labels per sample
- ☐ Model output: Multiple probabilities (sigmoid per class).
- ☐ Example: Image tagging (person, female, Kiwi)

5. More on Dataset

□ Dataset Size vs. Accuracy

□ Larger Datasets Improve Accuracy

□ More training data generally leads to higher classification accuracy.

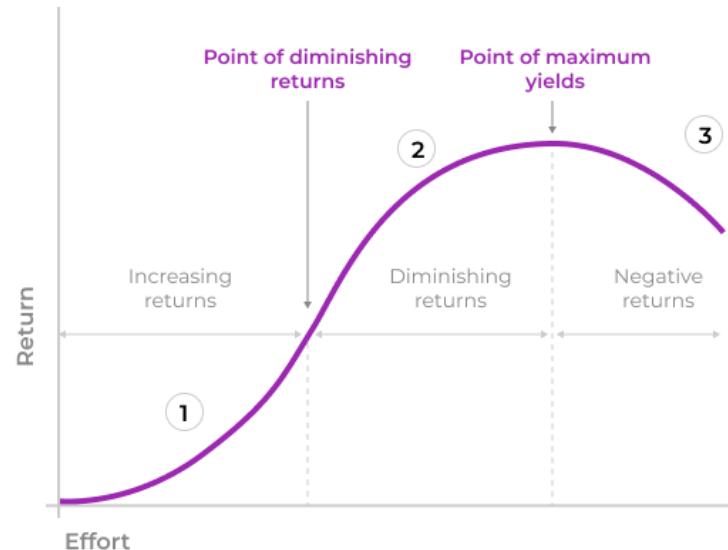
→ Better to have larger dataset always??

□ Computational Cost

□ Larger datasets require more processing power and training time.

□ Diminishing Returns

□ Accuracy gains slow down as dataset size increases.



5. More on Dataset

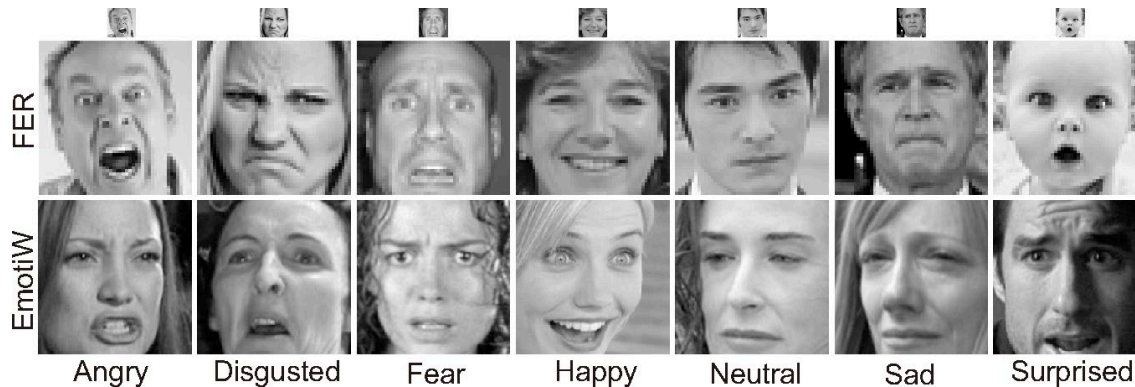
□ Dataset Size vs. Accuracy

□ Model Sensitivity

- May have different results with different models.

□ Difference from Datasets

- Labelling may be different from Datasets.



□ Quality Matters

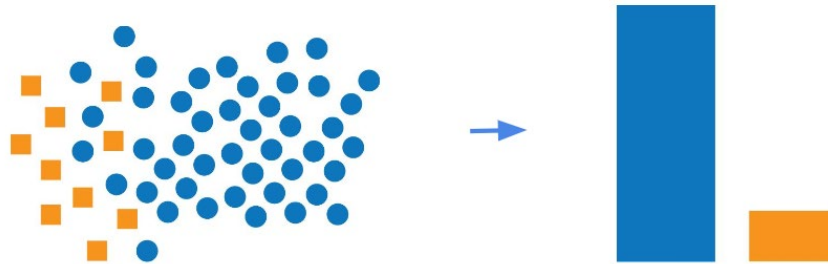
- Noisy or poorly labelled data can reduce accuracy, even with a large dataset.

5. More on Dataset

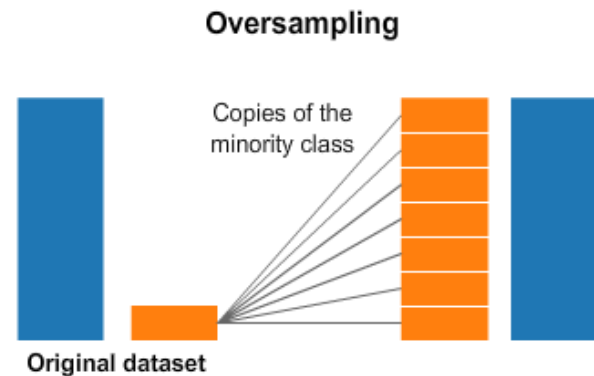
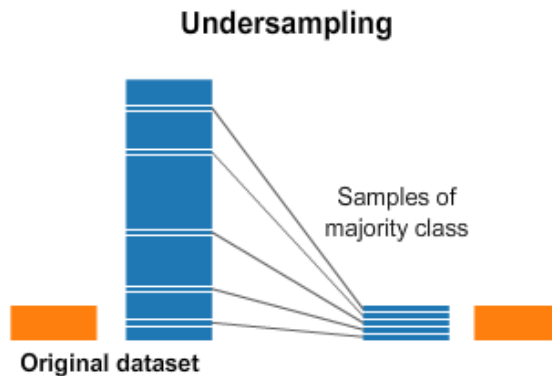
□ Dataset Size vs. Accuracy

□ Imbalanced Data

□ Increasing dataset size may not help minority classes without balancing techniques.



□ Solution?



6. Data Augmentation

☐ Data Augmentation

☐ What is Data Augmentation?

- ☐ Technique to artificially expand the training dataset.
- ☐ Helps improve model generalization and reduce overfitting.

☐ Why Use Data Augmentation?

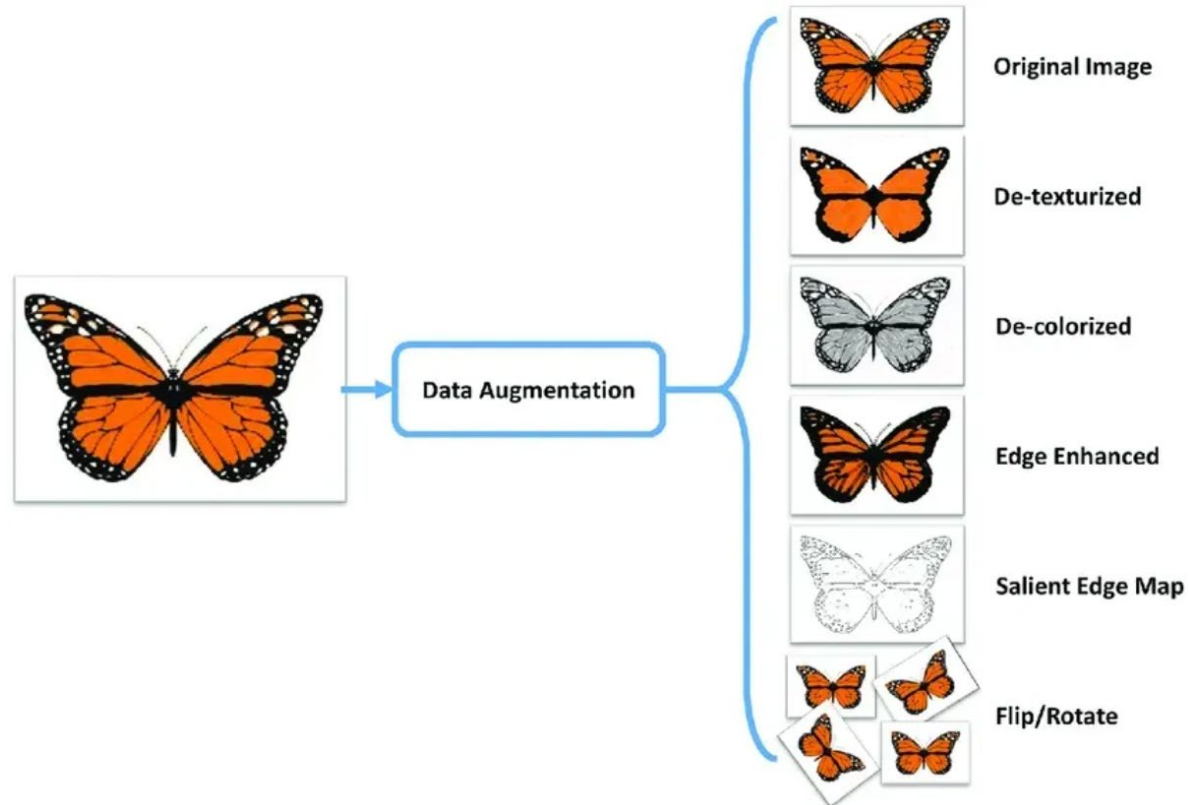
- ☐ Increases dataset diversity.
- ☐ Reduces overfitting, improving generalization.
- ☐ Enhances model robustness to real-world variations.
- ☐ Useful for small datasets.
- ☐ Sometimes can help, but augmentations can hurt accuracy

6. Data Augmentation

□ Data Augmentation

□ Common Augmentation Techniques

- Image Data: Rotation, flipping, cropping, scaling, brightness adjustment.

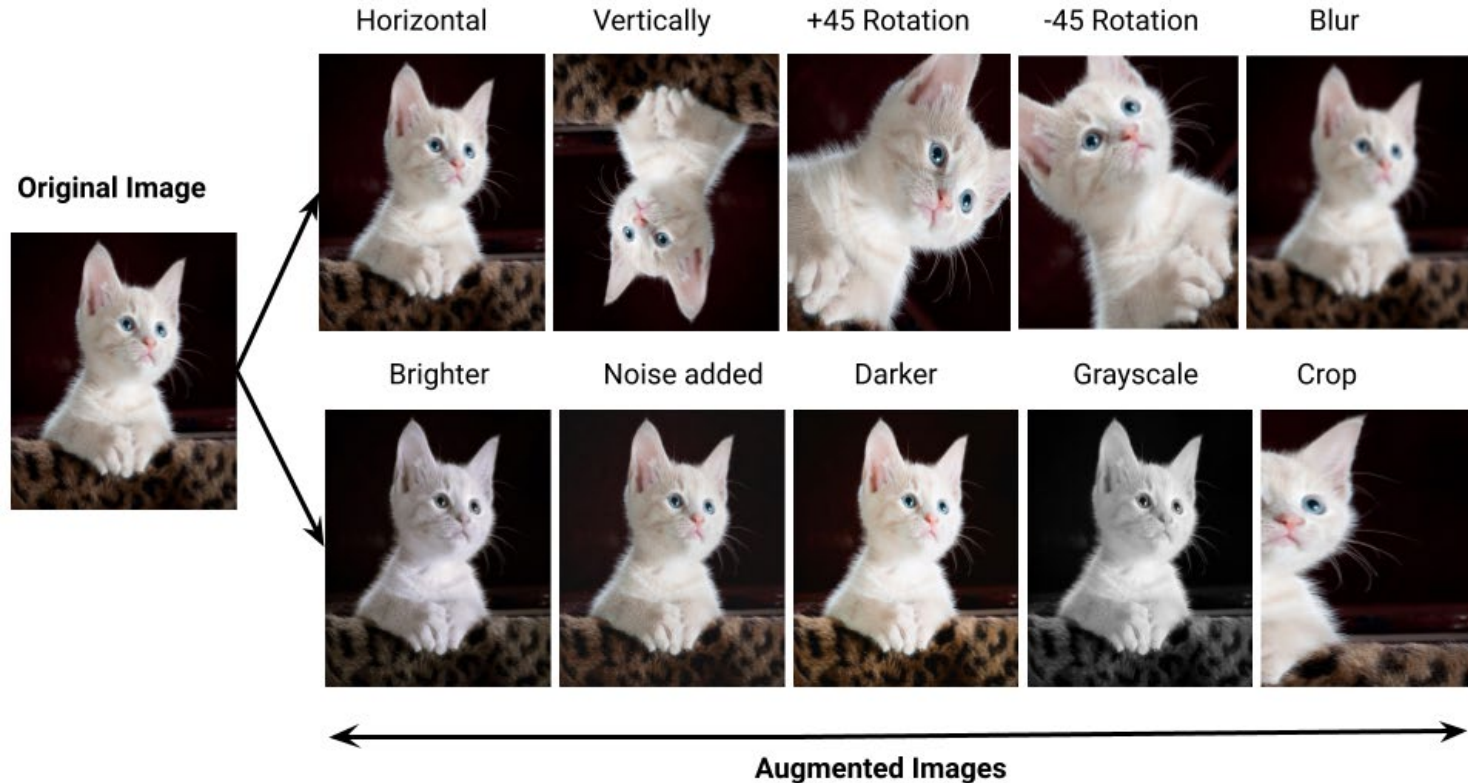


6. Data Augmentation

□ Data Augmentation

□ Common Augmentation Techniques

□ Image Data: Rotation, flipping, cropping, scaling, brightness adjustment.

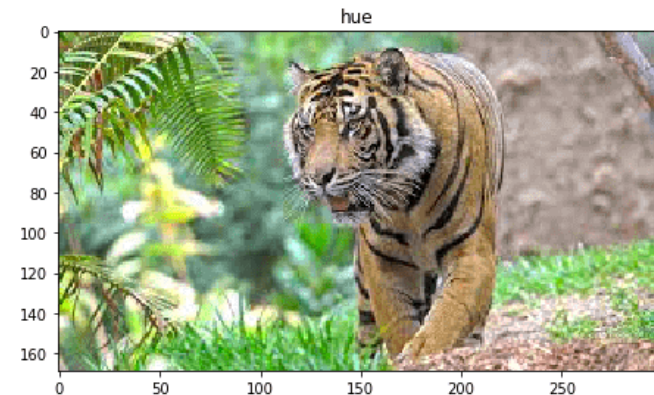
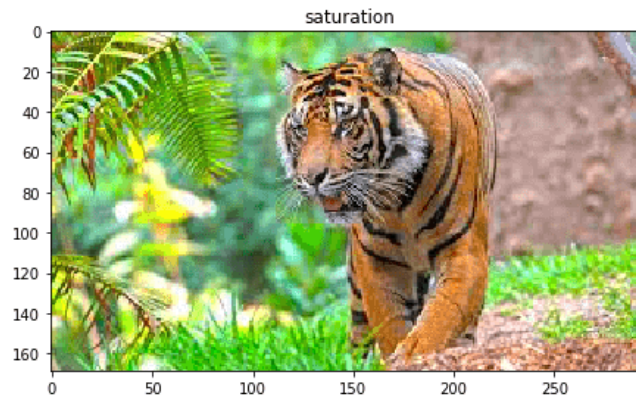
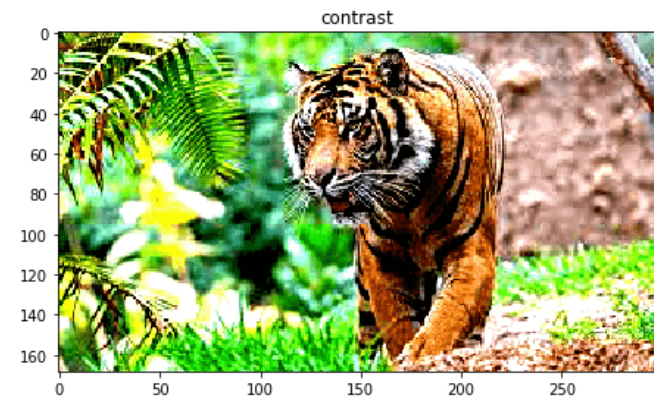
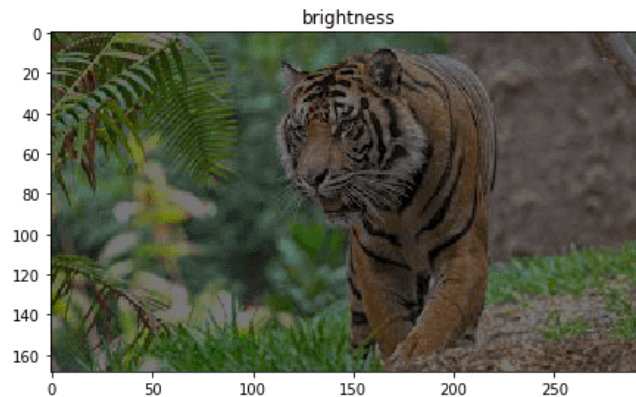


6. Data Augmentation

□ Data Augmentation

□ Common Augmentation Techniques

□ Image Data: Rotation, flipping, cropping, scaling, brightness adjustment.



6. Data Augmentation

□ Data Augmentation

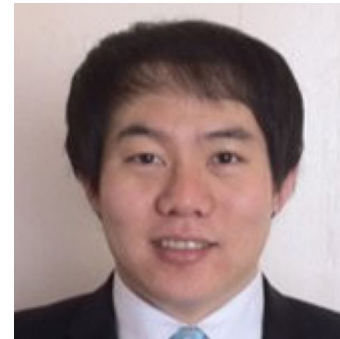
□ Pytorch transforms

- `transforms.Compose([ ])`: Chains multiple image transformations sequentially.
- `RandomResizedCrop(224, scale=(0.8, 1.0))`: Randomly crops and resizes the image to 224×224, keeping 80-100% of the original size.
- `RandomHorizontalFlip(p=0.5)`: Flips the image horizontally with a **50% probability**.
- `RandomRotation(degrees=30)`: Rotates the image randomly **within ±30 degrees**.
- `ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1)`: Randomly modifies image **brightness, contrast, saturation, and hue**.
- `ToTensor()`: Converts the image into a **PyTorch tensor** (normalizing pixel values to [0,1]).

```
import torch
import torchvision.transforms as transforms

# Define a sequence of augmentations
transform = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=30),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),
    transforms.ToTensor(),
])
```

Original Image



Augmented Image

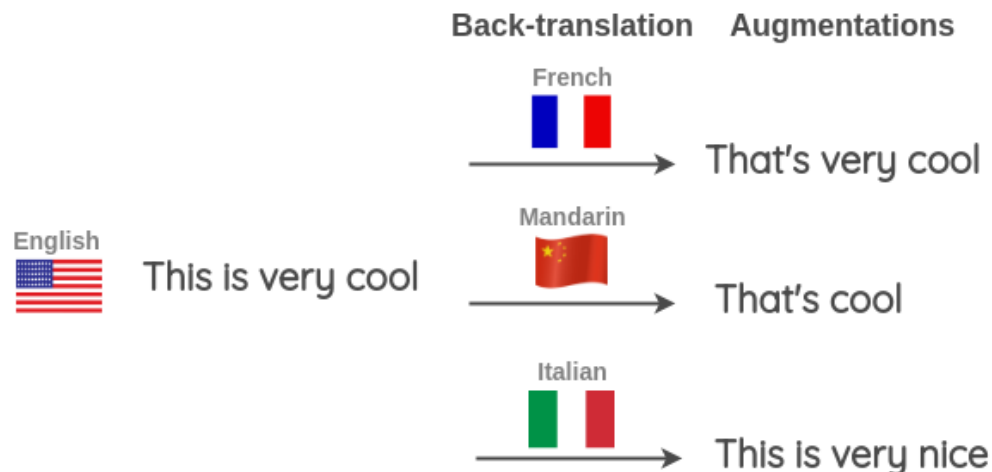
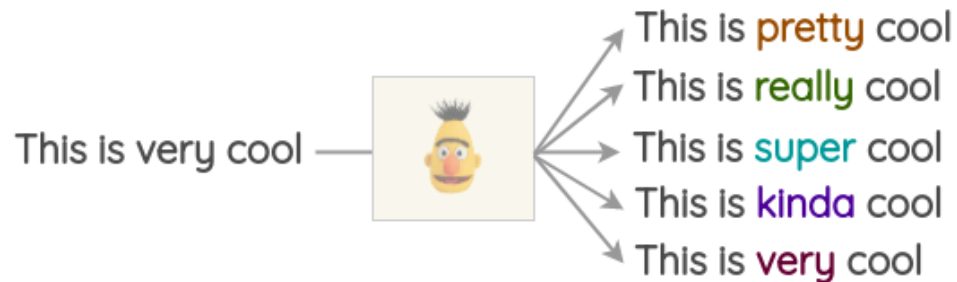


6. Data Augmentation

□ Data Augmentation

□ Common Augmentation Techniques

- Text Data: Synonym replacement, back translation, word shuffling.

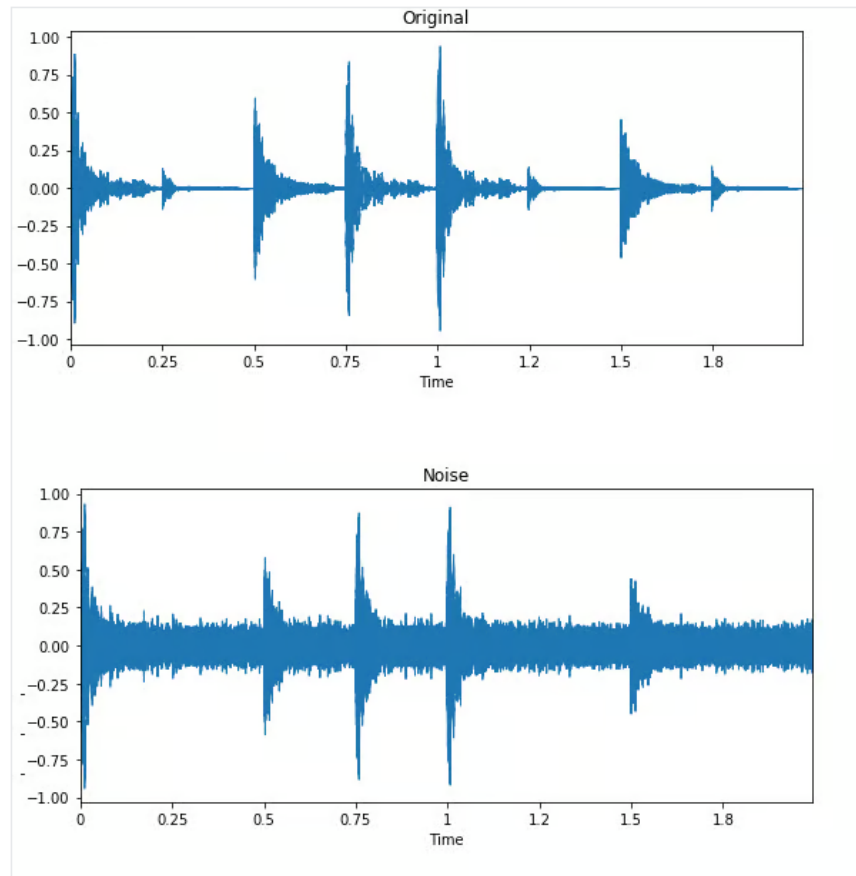


6. Data Augmentation

□ Data Augmentation

□ Common Augmentation Techniques

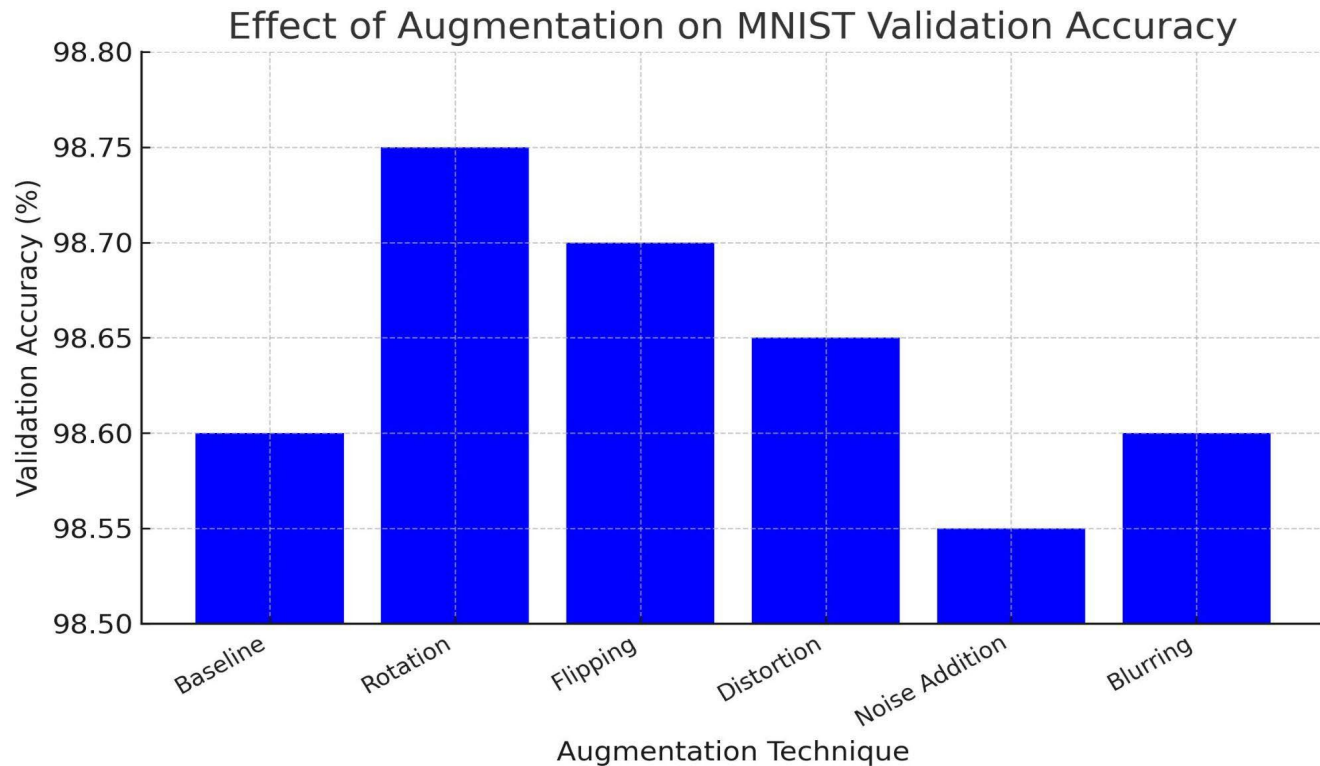
□ Audio Data: Time stretching, pitch shifting, noise addition.



6. Data Augmentation

□ Data Augmentation

□ Enhancing accuracy

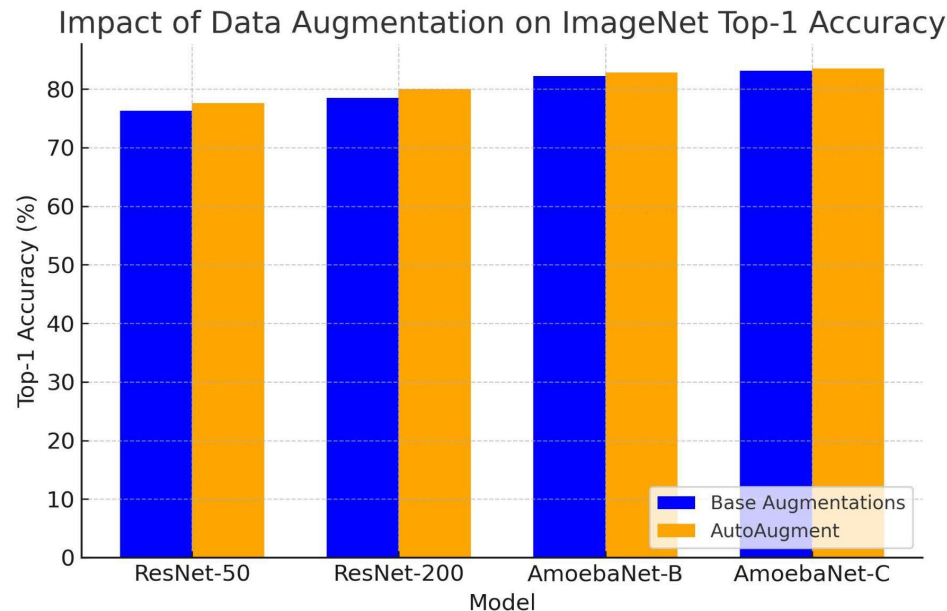


6. Data Augmentation

□ Data Augmentation

□ AutoAugment Improves Accuracy:

- All models show an accuracy boost with AutoAugment compared to base augmentations.

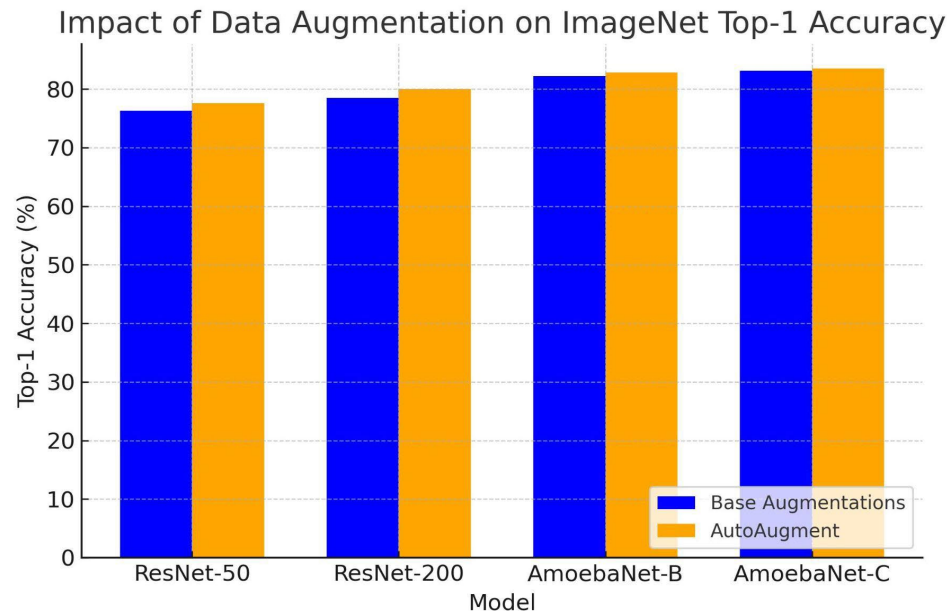


6. Data Augmentation

□ Data Augmentation

□ ResNet-50 and ResNet-200:

- ResNet-50: Accuracy improves from 76.3% → 77.6% (+1.3%).
- ResNet-200: Accuracy improves from 78.5% → 80.0% (+1.5%).



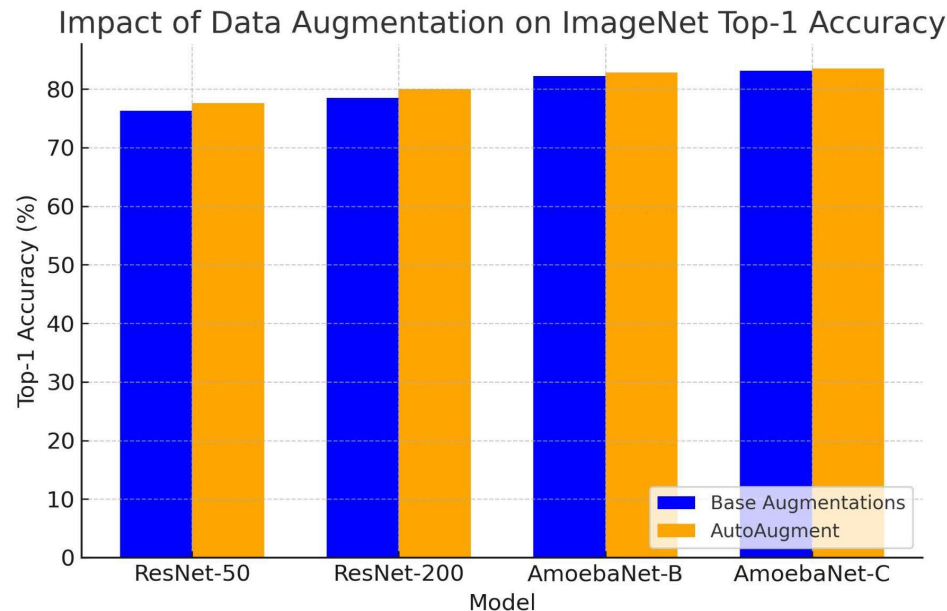
6. Data Augmentation

□ Data Augmentation

□ AmoebaNet Models Show Smaller Gains:

□ AmoebaNet-B: 82.2% → 82.8% (+0.6%).

□ AmoebaNet-C: 83.1% → 83.5% (+0.4%).



6. Data Augmentation

☐ **Data Augmentation**

☐ **Make sure...**

- ☐ Your augmentations should not change the data so much that your classes become unrecognizable.
- ☐ You visualize your augmentations and check it.

☐ **Which augmentations should you use?**

- ☐ If your model is performing well already - None
 - ☐ If your model is struggling - try add some simple augmentations
 - ☐ If your model is still performing poorly - try advance augmentations
- There is no “right” augmentation! It depends on your dataset and model!!!

7. Dataset Splits

□ Dataset Splits

□ Training Set

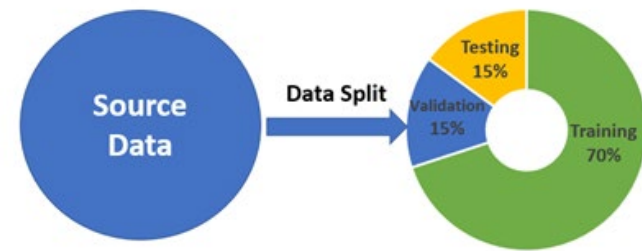
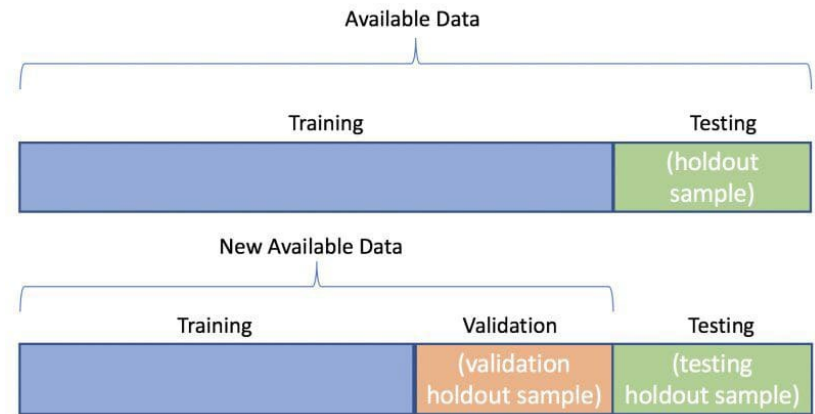
- Largest split ($\sim 70\text{-}80\%$ of data).
- Used for model learning and parameter updates.

□ Validation Set

- Smaller split ($\sim 10\text{-}20\%$ of data).
- Used for hyperparameter tuning and early stopping.

□ Test Set

- Final split ($\sim 10\text{-}20\%$ of data).
- Evaluates generalization on unseen data.

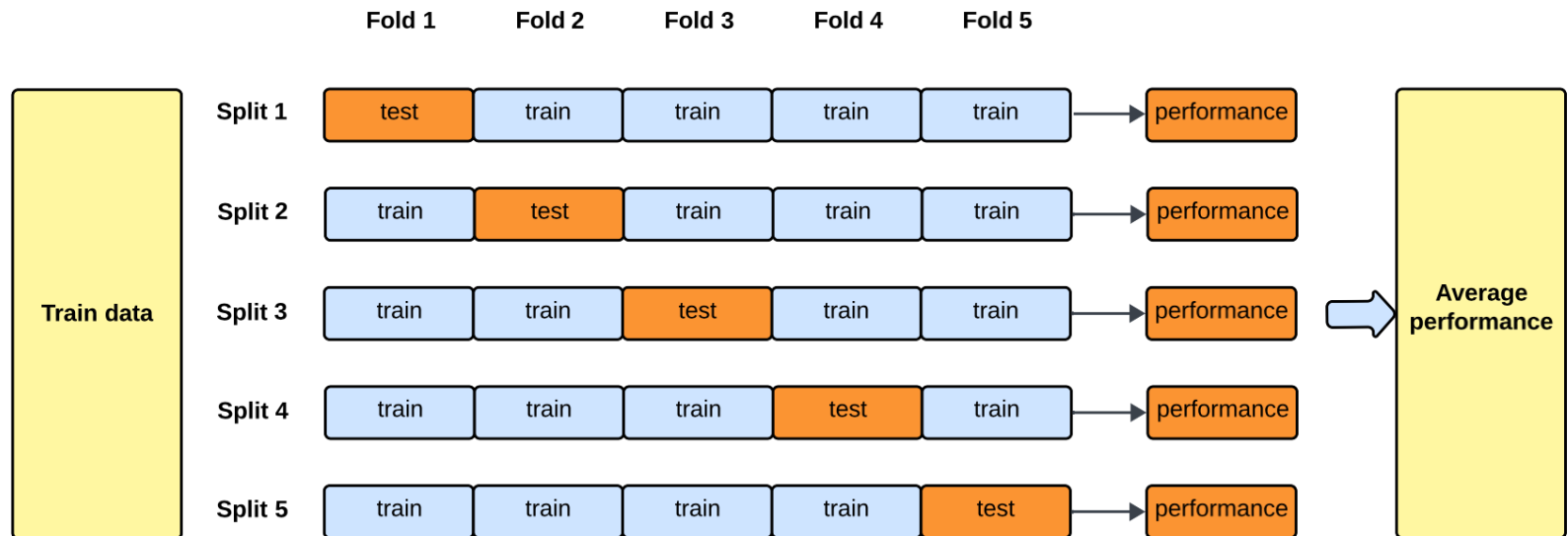


7. Dataset Splits

□ Dataset Splits

□ Training and Test Sets

□ Usually 80% - 20%.

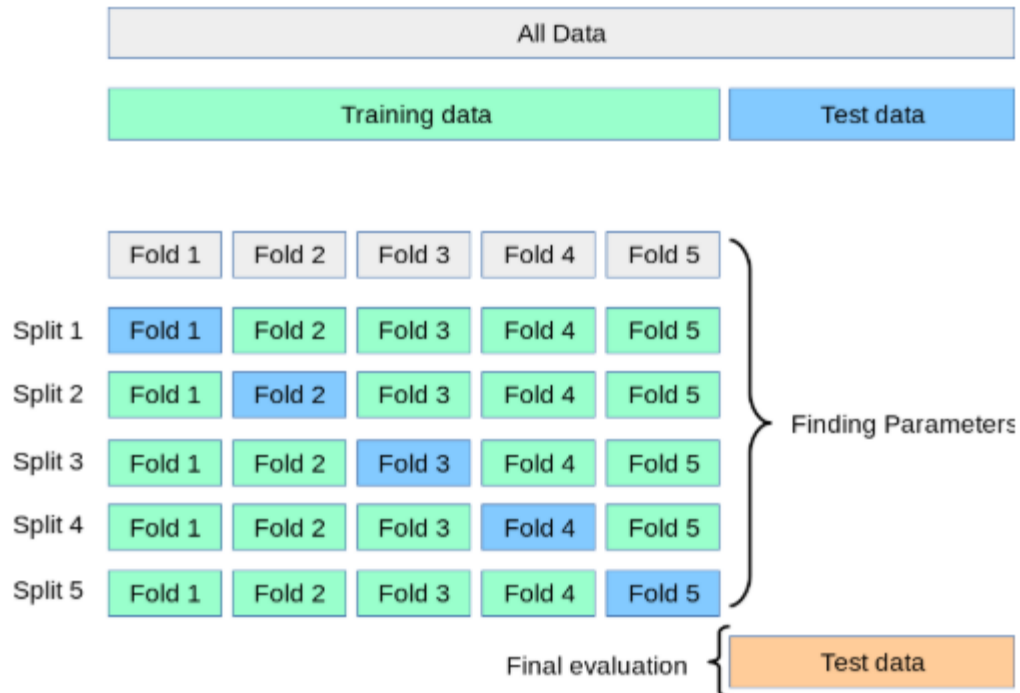


7. Dataset Splits

□ Dataset Splits

□ Training, Validation and Test Sets

□ Usually 60% - 20% - 20%.



8. Summary

☐ **More training or data/augmentation needed?**

☐ **If your model is undertrained:**

- ☐ If you have only trained for a few epochs and your validation accuracy is still improving, **continue training for longer** before making other adjustments.

☐ **If validation accuracy is stuck after many epochs:**

- ☐ If you have trained for a large number of epochs but validation accuracy is not improving, **try adding more data augmentation** to enhance generalization.

☐ **If multiple augmentations do not help:**

- ☐ If you have already applied multiple augmentations and validation accuracy is still not improving, you **may need more data** to improve performance.

☐ **Training strategy recommendations:**

- ☐ Aim to **train your model first** until it converges (i.e., stops improving) before introducing additional augmentations.
- ☐ Try **adding augmentations** before increasing the dataset size to maximize data efficiency.

8. Summary

- ❑ **Dataset Quality Matters** – Labels must be accurate to avoid training on incorrect information.
- ❑ **Dataset Size vs. Accuracy** – More data generally improves accuracy, but with diminishing returns.
- ❑ **Proper Dataset Splits** – Train (~70-80%), validation (~10-20%), and test (~10-20%) ensure reliable evaluation.
- ❑ **Data Augmentation Helps** – Increases diversity, reduces overfitting, and improves generalization.
- ❑ **Common Augmentations** – Rotation, flipping, cropping, scaling, brightness adjustment, noise.
- ❑ **Advanced Techniques Boost Performance** – MixUp, CutMix, GridMask, AutoAugment, and AugMix.
- ❑ **Augmentation vs. Accuracy Trade-off** – Helps but may not always improve performance; requires tuning.
- ❑ **No Universal Best Augmentation** – Effectiveness depends on dataset, model, and task requirements.
- ❑ **Experimentation is Key** – Test different augmentations to find the best combination for your model.